Использование LLM

Короткий ответ: **платформа полностью работает без LLM и без ключей** (так работает демо-стенд жюри). Языковая модель — необязательная надстройка в двух местах разбора текста и один офлайн-инструмент авторов банка заданий. В тесте, оценке, ранжировании и объяснениях её нет и быть не может.

Организаторы разрешили облачные LLM (в том числе зарубежные; предпочтительны открытые модели) при условии, что команда документирует, какие модели, как и зачем используются, и берёт на себя риски 152-ФЗ, если ПДн уходят в модель. GPU на стенде жюри нет, поэтому локального инференса в поставке нет: модель — всегда внешний API.

1. Где LLM используется

#	Место	Что делает модель	Режим по умолчанию	Если модели нет
1	Разбор брифа работодателя (`app/services/matching/llm_brief.py` , задача `brief_skills`)	Только **добавляет** навыки, которые пропустили правила разбора; вид навыка всегда «желательный»	Выключено	Бриф разбирают правила; результат тот же, что в демо
2	Разбор резюме кандидата (`app/services/resume/parser.py` , задача `resume`)	Только **добавляет** навыки, роли, soft skills, заявленный грейд, которых не нашли правила	Выключено	Работают правила
3	Офлайн, у авторов банка: черновики генераторов семейств заданий	«LLM пишет генератор, а не задание» (см. §4)	Не часть продукта	—

Общие гарантии для пунктов 1 и 2 (шлюз `app/llm/gateway.py`):

- **Только добавление и только с цитатой.** Каждое поле от модели обязано прийти с дословной цитатой из текста (`rapidfuzz.partial_ratio ≥ 90` , `VZ_LLM_MIN_QUOTE_RATIO`). Поле без цитаты, с чужим названием или вне справочника навыков отбрасывается. Результат правил модель изменить не может.
- **Чипы помечены источником**: «правило», «вручную» или имя провайдера; для резюме все элементы — серые «заявлено», подтверждением не считаются.
- **Детерминированный fallback.** Цепочка провайдеров → «без LLM»: сбой, таймаут (общий бюджет 20 с на разбор), невалидный JSON или промах кассеты дают тот же результат, что и выключенная модель. Исключения провайдера в разбор не пробрасываются.
- **Воспроизводимость.** Кэш по `sha256(версия промпта + задача + замаскированный текст)` . Режимы `VZ_LLM_MODE` : `record` (провайдеры плюс запись ответов в JSON-кассету), `replay` (только кассета, без сети и ключей — демо с LLM воспроизводимо). В кассете нет текста запроса, только ключ `sha256` и ответ модели на замаскированный текст.
- **Температура 0** у всех провайдеров (у GigaChat 0,01: строгий 0 он не принимает).

2. Где LLM запрещена и почему

Запрещена в: выдаче и ключах заданий, сложности `b` , адаптивном выборе и остановке теста, определении грейда и категории, Score, ранжировании подборки, объяснениях «почему выше», вердиктах «Антиподсказчика», раскрытии контактов.

Причины:

1. **Сопоставимость.** Результат теста должен быть сравним между людьми и между попытками. Задание «под каждого» от модели не имеет известной сложности, а ключи модели ошибаются: в симуляции каждые 5 % битых ключей заметно снижают долю верных решений (VALIDATION.md §2.5; долю ошибок у реальных моделей мы не измеряли). Ключи считает исполнение кода генератора, не модель.

2. **Воспроизводимость и аудит.** Подборка и объяснения обязаны быть повторяемыми и проверяемыми (в Score «сумма вкладов = итог» проверяется тестом); недетерминированная модель в этой цепочке разрушает проверяемость.
3. **152-ФЗ, ст. 16.** Решение о человеке не принимается автоматически без человека; модель не должна влиять на то, кого показывают работодателю.
4. **Резюме не влияет ни на категорию, ни на Score** (правило продукта): даже то, что модель извлекла из резюме, остаётся серым «заявлено».

Как это защищено в коде и тестах:

- Модули Score, ранжирования, объяснений не импортируют `app.llm`: `tests/test_api_needs_review.py::test_ranking_path_never_imports_llm`; в `app/testing` и `app/services/testing` импортов `app.llm` нет.
- **Поведенческий тест** `tests/test_llm_boundaries.py`: шлюз подменён на «включённый» с записью каждого обращения (и провайдеры тоже перехвачены). Полная сессия адаптивного теста с результатом, сборка версии подборки, выдача, карточка кандидата — **0 обращений**. Единственное допустимое обращение на стороне работодателя — `brief_skills` при разборе брифа.

3. Какие модели

Шлюз поддерживает три вида провайдеров; цепочка при `VZ_LLM_PROVIDER=auto` идёт в порядке ниже, неподключённые пропускаются.

Провайдер	Значение `VZ_LLM_PROVIDER`	Модель по умолчанию	Для чего	Юрисдикция
YandexGPT (Foundation Models API)	`yandexgpt`	`yandexgpt-lite/latest`	Основной для боевой установки: данные остаются в РФ	РФ
GigaChat	`gigachat`	`GigaChat` (scope `GIGACHAT_API_PERS`)	Проверен вживую (§9). TLS проверяется по корневому сертификату НУЦ Минцифры из поставки (`VZ_LLM_GIGACHAT_VERIFY=bundled`, умолчание)	РФ
OpenAI-совместимый API	`openai_compat`	`qwen/qwen3-235b-a22b-2507` (меняется `VZ_LLM_OPENAI_MODEL`)	**Открытые модели** (Qwen, Llama, DeepSeek, gpt-oss) у облачных хостингов — OpenRouter, Together, Fireworks и им подобных; собственный vLLM/Ollama, если появится GPU; при необходимости зарубежные API	Зависит от хостинга

Зачем именно эти: задача модели узкая (извлечь названия технологий с цитатой из короткого текста), поэтому нужна не мощная, а дешёвая и быстрая модель с хорошим русским; точность держит проверка цитаты, а не размер модели. Открытая модель через OpenAI-совместимый адрес даёт предпочтительный организаторами вариант без GPU на нашей стороне и без привязки к вендору.

TLS для GigaChat

Серверы Сбера (`ngw.devices.sberbank.ru:9443`, `gigachat.devices.sberbank.ru`) подписаны цепочкой НУЦ Минцифры, которой нет в стандартных хранилищах. В поставке лежит корневой сертификат `backend/app/llm/certs/russian_trusted_root_ca.pem` («Russian Trusted Root CA», действует до 27.02.2032), проверка включена всегда:

- SHA-256: `D2:6D:2D:02:31:B7:C3:9F:92:CC:73:85:12:BA:54:10:35:19:E4:40:5D:68:B5:BD:70:3E:97:88:CA:8E:CF:31` (SHA-1 `8F:F9:15:CC:AB:7B:C1:6F:8C:5C:80:99:D5:3E:0E:11:5B:3A:EC:2F`). Файл скачан с официального CDN госуслуг (`gu-st.ru`, страница gosuslugi.ru/crt) двумя путями (отдельный файл корня и официальный

комплект `russiantrustedca.pem`) — отпечатки совпали; отпечаток закреплён тестом `tests/test_llm_gigachat.py`. Сам текст отпечатка на странице госуслуг читается только скриптом, поэтому сверка с «опубликованным значением» сделана криптографически: подчинённый сертификат подписан этим корнем, а цепочка живых серверов Сбера (`openssl s_client -CAfile`) проверяется с кодом 0 для обоих хостов.
- `VZ_LLM_GIGACHAT_VERIFY`: `bundled` (умолчание) — сертификат из поставки; путь к PEM — свой; `true` — системное хранилище; `false` — без проверки, **только для разработки**, в боевой установке не использовать.

Модели, доступные личному scope `GIGACHAT_API_PERS` (список `GET /api/v1/models`, 09.10.2026): `GigaChat`, `GigaChat-2`, `GigaChat-2-Pro`, `GigaChat-2-Max`, `GigaChat-Pro`, `GigaChat-Max`, `GigaChat-Plus`, а также `-preview`-версии; эмбеддинги нам не нужны. Выбор модели — `VZ_LLM_GIGACHAT_MODEL`; по результатам §9 оставлена `GigaChat`.

4. Офлайн-использование: «LLM пишет генератор, а не задание»

Авторы банка заданий могут использовать LLM как помощника при написании **кода семейства** (шаблон текста, генератор параметров, эталонный решатель). Это инструмент разработки, не часть продукта и не вызывается рантаймом. Правила:

- Ключи ответов считает **исполнение** решателя, а не модель; черновик проходит автоматические гейты банка (`app/testing/bank/base.py`, тесты `tests/test_bank_*`: идемпотентность клонов, проверка ключей исполнением, отсутствие утечек) и ревью человека.
- Затем pretest с весом 0 и калибровка по ответам людей; без измеренной сложности семейство в оценку не входит.
- Автоматического конвейера «фабрика семейств» в репозитории **нет** (в TESTING_MECHANISM.md §13 он числится в плане): черновики генераторов писались с помощью ассистента и прошли ту же проверку исполнением и ревью. Никаких ПДн в этом процессе нет.

5. Политика ПДн

Что **не уходит** в модель никогда: контакты и ПДн кандидата из профиля, результаты тестов, ответы на задания, Score, данные ФСП, содержимое подборок и приглашений. Эти данные в шлюз не передаются: он вызывается только с текстом брифа (задача `brief_skills`) и текстом резюме (задача `resume`).

Что уходит и как защищено:

Текст	Что в нём	Меры
Бриф работодателя	Описание вакансии: стек, вилка, формат. ПДн кандидатов нет; возможны имена контактных лиц компании	Маскирование тем же модулем; только задача `brief_skills`
Резюме кандидата	Может содержать ПДн	**Маскирование до вызова** (`app/llm/masking.py`): email, телефоны, ссылки на профили и Telegram, строки с датой рождения/возрастом, ФИО, найденные правилами, заменяются на `*` с сохранением длины (смещения цитат совпадают с исходным текстом); `extra_pii_spans` добавляет найденные ФИО. Текст обрезается до `VZ_LLM_MAX_INPUT_CHARS` (12 000)

Остаточный риск, который берёт на себя команда: маскирование основано на правилах, поэтому ФИО, не распознанные правилами, и косвенные идентификаторы (названия мест работы, должности с датами) в замаскированном тексте могут остаться. Поэтому:

1. **По умолчанию LLM выключена**; в демо-стенде и у жюри текст резюме и брифа никуда не уходит.

2. **Российские провайдеры (YandexGPT, GigaChat) — для боевой установки с резюме**: данные не покидают РФ (ст. 18 ч. 5 152-ФЗ).
3. **Зарубежный/внешний OpenAI-совместимый адрес по умолчанию получает только задачу `brief_skills`** (`VZ_LLM_OPENAI_TASKS=brief_skills`). Задача `resume` уйдёт туда только если оператор явно добавит её в `VZ_LLM_OPENAI_TASKS` ; тогда он обязан оценить трансграничную передачу (ст. 12) и получить согласие кандидата. Это защищено тестом (`test_openai_compat_provider_and_task_restriction`).
4. **Известное ограничение:** отдельной цели «обработка резюме языковой моделью» в журнале согласий (`ConsentPurpose`) пока нет. Поэтому для боевой установки с включённым разбором резюме моделью её нужно добавить (отдельная галочка с версией текста) или ограничиться уведомлением в согласии на обработку; в демо и у жюри функция выключена. Секреты ключей — `SecretStr` , в логи не пишутся.

6. Как включить

Всё через окружение (`.env.example` , раздел «LLM»). Без ключей и без изменений ничего не вызывается.

```
# Российский провайдер (боевая установка)
VZ_LLM_PROVIDER=yandexgpt
VZ_LLM_YANDEX_API_KEY=...        # ключ сервисного аккаунта
VZ_LLM_YANDEX_FOLDER_ID=...

# Открытая модель через OpenAI-совместимый хостинг (брифы; резюме не уходит)
VZ_LLM_PROVIDER=openai_compat
VZ_LLM_OPENAI_BASE_URL=https://openrouter.ai/api/v1
VZ_LLM_OPENAI_API_KEY=...
VZ_LLM_OPENAI_MODEL=qwen/qwen3-235b-a22b-2507

# Цепочка: YandexGPT -> GigaChat -> OpenAI-совместимый -> без LLM
VZ_LLM_PROVIDER=auto

# Демо с «включённой» LLM без сети и ключей (кассета записывается один раз через VZ_LLM_MODE=record)
VZ_LLM_MODE=replay
VZ_LLM_CASSETTE_PATH=/app/data/llm-cassette.json
```

Признак для проверяющего: в ответе разбора брифа `llm_used` и поле `source` чипов («YandexGPT» / «GigaChat» / `openai_compat`); при выключенной модели `llm_used=false` , все чипы «правило» или «вручную».

7. Стоимость и ресурсы

Нагрузка на модель мала: один вызов на разбор брифа или резюме, кэш по `sha256` убирает повторы, правила отрабатывают первыми. Порядок величин для оценки бюджета (тарифы провайдеров меняются, актуальные проверять перед включением):

Операция	Токены (вход / выход)	Комментарий
Бриф	≈ 0,5–1 тыс. / ≈ 0,2 тыс.	Один вызов на версию текста
Резюме	≈ 1–3 тыс. / ≈ 0,3 тыс.	Один вызов на загрузку, вход ограничен 12 000 символов

Для десятков разборов в день, как у демо и пилота, это единицы рублей в сутки у российских провайдеров и копейки у открытых моделей на хостингах. Лимиты запросов к API кандидата и работодателя ограничивают число вызовов сверху. GPU, собственного инференса и дополнительных сервисов не требуется; без ключей затрат нет.

8. Как проверить

```
cd backend
uv run pytest -q tests/test_llm_boundaries.py tests/test_resume_llm.py
tests/test_api_needs_review.py::test_ranking_path_never_imports_llm
```

Тесты проверяют: умолчание «выключено» (ни провайдеров, ни обращений), 0 обращений в сессии теста и ранжировании, ограничение задач зарубежного провайдера и маскирование ПДн в запросе, деградацию в «без LLM» при ошибке провайдера, цитатную проверку и кассету.

9. Результаты на отложенной выборке брифов (GigaChat, 09.10.2026)

Выборка: 62 синтетических брифа `backend/eval/brief_heldout.jsonl` (ПДн нет). Запуск: `uv run python -m eval.llm_brief_eval --mode record|replay|live`. «Правила» — `parse_brief`; «+LLM» — те же правила плюс `llm_brief.augment` (LLM добавляет «желательные» навыки с цитатой). Модель основного прогона и кассеты — `GigaChat` (scope `GIGACHAT_API_PERS`), температура 0,01, `VZ_LLM_MIN_QUOTE_RATIO=90`.

Метрика	Только правила	Правила + LLM
F1 навыков (все)	0,975 (P 0,976 / R 0,973)	0,975 (P 0,976 / R 0,973)
F1 must	0,842 (P 0,984 / R 0,736)	0,842 (без изменений)
Специализация top-1	0,984	0,984
Грейд	1,000	1,000
Вилка (точно) / основание	0,952 / 0,972	0,952 / 0,972
Формат / город	1,000 / 0,984	1,000 / 0,984

Честный вывод: на этой выборке LLM ничего не улучшает. Правила уже находят 97 % навыков; из 348 полей, предложенных моделью, 321 уже найдено правилами, 7 отсутствуют в справочнике навыков, 20 отброшено охранником цитаты, ни одного нового навыка не добавлено. Из 10 навыков эталона, которые пропустили правила, модель не предложила ни одного. Сама по себе модель (без правил) даёт F1 0,908 (P 0,987 / R 0,840): надёжна по точности, но слабее словаря по полноте. LLM-путь остаётся страховкой на лексику вне словаря, а не источником качества, и по умолчанию выключен.

Другие модели того же scope (живые прогоны без кассеты, те же 62 брифа):

Модель	F1 навыков с LLM	Добавлено навыков (верных)	F1 одной модели	Задержка mean / p95, с	Токены вход / выход
GigaChat (запись кассеты)	0,975	0	0,908	1,0 / н.д.	5 082 / 8 288
GigaChat (повторный живой прогон)	0,975	0	0,914	1,1 / 1,8	5 116 / 8 377
GigaChat-2	0,975	0	0,912	1,0 / 1,7	5 151 / 8 422
GigaChat-Pro	0,973	5 (2)	0,928	2,7 / 4,3	4 825 / 8 081
GigaChat-2-Pro	0,973	5 (2)	0,930	2,6 / 3,8	4 974 / 8 065
GigaChat-2-Max	0,975	4 (2)	0,924	3,2 / 5,5	9 603 / 8 333
GigaChat-Max	0,975	4 (2)	0,924	3,2 / 5,8	9 829 / 8 264

Повторный прогон `GigaChat` дал чуть другое поле предложений (F1 одной модели 0,914 против 0,908 при записи): даже при температуре 0,01 ответы не бит-в-бит, поэтому воспроизводимость обеспечивает кассета, а не повтор вызова. Старшие модели добавляют 4–5 навыков, из них верны 2: полнота растёт на 0,6 п.п., точность

падает на 0,5–0,8 п.п., F1 не растёт, а задержка в 3 раза выше. Поэтому берём самую быструю `GigaChat`. (Pro-модели один раз вернули невалидный JSON — это штатный fallback «без LLM» для этого брифа.)

Охранник цитаты (`VZ_LLM_MIN_QUOTE_RATIO`, кассета `GigaChat`, повтор без сети):

Порог partial_ratio	Отброшено полей из 348	F1 одной модели (P / R)
100 (дословно)	39	0,878 (0,987 / 0,791)
90 (умолчание)	20	0,908 (0,987 / 0,840)
70	8	0,922 (0,985 / 0,866)
50	3	0,928 (0,985 / 0,877)

Отброшенные поля — в основном перифразы цитаты («питон» вместо «python»), а не выдумки: точность модели почти не зависит от порога. Значит, на этой выборке порог 90 стоит полноты, а не защищает точность; оставляем 90 как защиту от галлюцинаций на более длинных и шумных текстах (там цитата — единственная проверка), решение о снижении — после измерения на реальных брифах пилота.

Надёжность и стоимость. Сбоев и таймаутов нет: 62 запроса из 62 получили ответ (доля fallback 0,000); токен OAuth запрашивается раз в ~25 минут. Задержка `GigaChat`: mean 1,0–1,1 с, p95 1,8 с, максимум 2,3 с (в бюджете 20 с). Ограничение личного scope: два одновременных прогона по 62 запроса дали 429 (`HTTPStatusError`) на 7 и 18 запросах — шлюз штатно ушёл в «без LLM» (результат правил), но для боевой нагрузки нужен платный scope или очередь запросов; последовательные прогоны шли без единого сбоя. Расход: 5 082 входных + 8 288 выходных токенов на 62 брифа, то есть ≈ 82 + 134 токена на бриф; весь прогон — около 13,4 тыс. токенов (тариф Сбера для личного scope проверять в кабинете; порядок — копейки на прогон).

Что уходит в модель. Только текст брифа работодателя (после маскирования `app/llm/masking.py`) и фиксированная инструкция. Резюме, контакты, результаты тестов, Score не отправляются: задача `resume` идёт к GigaChat только если оператор сам включит разбор резюме (по умолчанию выключен, §5). Брифы выборки синтетические; реальные брифы — тексты работодателя без ПДн кандидатов (152-ФЗ: оператор данных работодателя — он сам; возможные имена контактных лиц компании маскируются правилами `masking.py` лишь частично). Ключ авторизации хранится только в окружении/ `.env` (вне git), в логи и кассеты не попадает.

Воспроизведение без ключа и сети. Кассета ответов `backend/eval/llm_cassette_brief.json` (62 записи: ключ `sha256` и ответ модели; текст запроса не хранится) лежит в репозитории:

```
cd backend
uv run python -m eval.llm_brief_eval --mode replay            # метрики таблицы выше, детерминированно
# стенд: шлюз берёт ответы из кассеты, ключи не нужны
VZ_LLM_PROVIDER=gigachat VZ_LLM_MODE=replay VZ_LLM_CASSETTE_PATH=eval/llm_cassette_brief.json
# перезапись кассеты вживую (нужен VZ_LLM_GIGACHAT_AUTH_KEY)
uv run python -m eval.llm_brief_eval --mode record
```

Промах кассеты (новый текст брифа) — «без LLM»: результат правил. Тесты: `tests/test_llm_gigachat.py` (отпечаток сертификата, форма OAuth/чат-запросов, воспроизведение кассеты), границы — `tests/test_llm_boundaries.py` прогонялись и с живой конфигурацией (`VZ_LLM_PROVIDER=gigachat`, ключ в окружении): тест, оценка и ранжирование — 0 обращений к шлюзу и провайдерам.